

Sistemas de Informação Distribuídos

Projecto
Labirinto

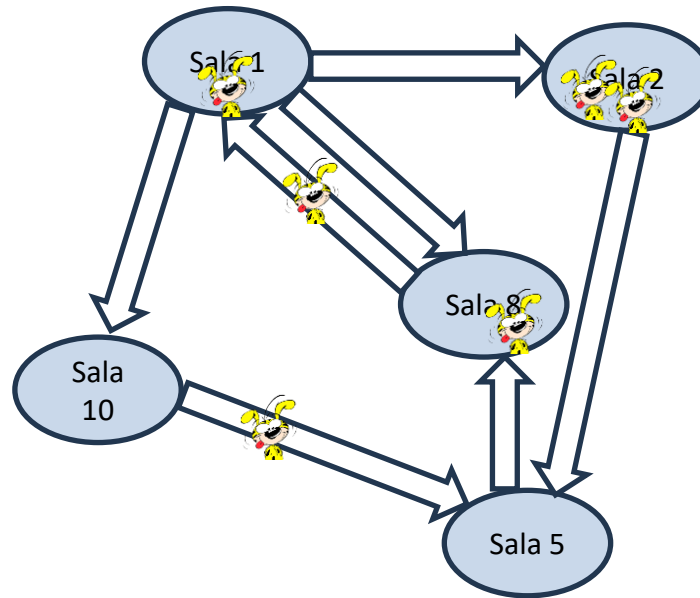
Objetivo Projeto

Depois de terminado o projeto os alunos terão experimentado implementar uma solução que contempla:

- Dados distribuídos em pelo menos dois computadores;
- Dados armazenados em duas tecnologias diferentes: modelo relacional (Mysql) e não relacional (MongoDB);
- Tolerância a falhas originadas por “crash” em bases de dados/pcs e em software;
- Migração incremental de dados entre bases de dados;
- Utilização de protocolo MQTT para recepção de dados de sensores e transmissão de dados entre diferentes pcs;
- Detecção de outliers e dados “sujos” provenientes de sensores;
- Atuação em tempo real no sistema (granularidade de segundo);
- Utilização das linguagens python, php e android studio;
- Lidar com indefinições de requisitos.

Labirinto

- Salas ligadas por corredores unidirecionais (só se caminha num sentido);
- A uma sala podem chegar e sair vários corredores;
- Os marsamis começam por ser largados numa sala e movem-se livremente (aleatoriamente?) pelos corredores;
- Os corredores podem ser individualmente fechados e abertos (impedindo marsamis de circular);
- Os marsamis cansam-se, passado um determinado tempo eles deixam de se deslocar;
- Os marsamis imobilizam-se definitivamente caso temperatura ou nível de ruído do labirinto atinja um certo valor;
- Existem marsamis “odd” e “even”.



Programa (“Simulador”)

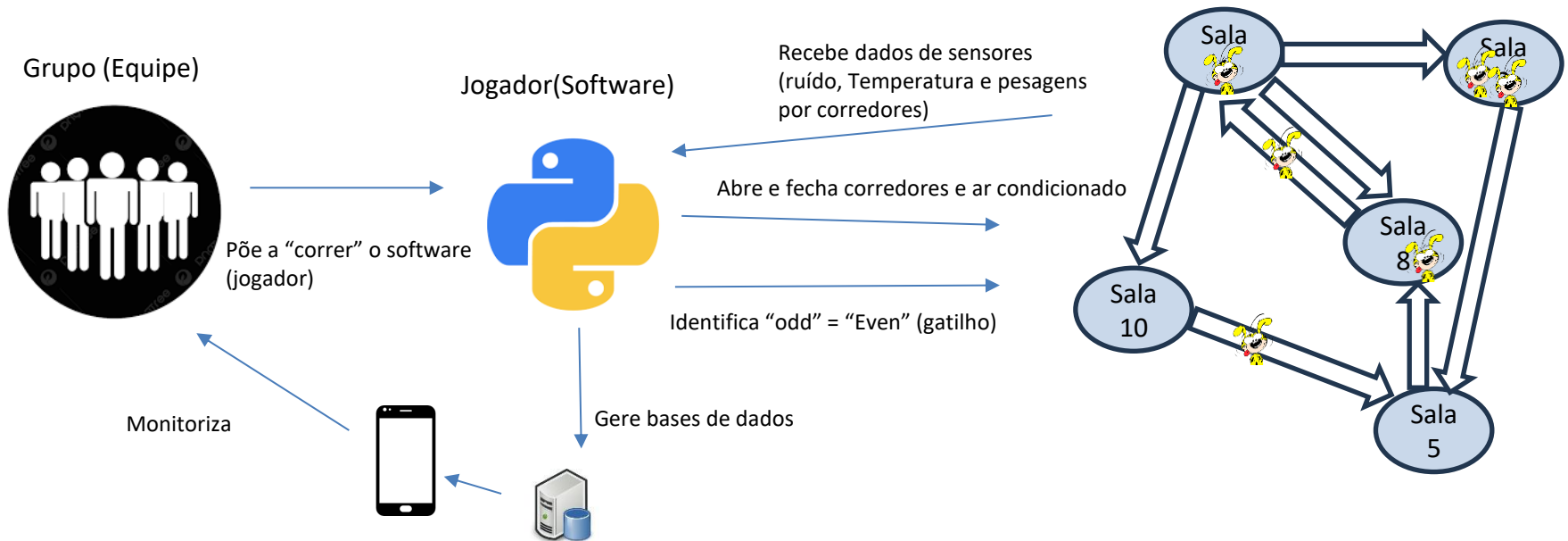
O “simulador” inicia com a largada dos marsamis nas salas (distribuídos pelas salas)

O “simulador” termina quando uma das seguintes situações ocorre:

- Foi atingido o limite de temperatura ou ruído;
- Decorreu um determinado período de tempo (pré definido pelo servidor) sem que nenhum marsami se mova (estão todos “cansados”, ver mais adiante status das mensagens “Cansado”).

O jogador (software) pretende maximizar o número de pontos, sendo que pode obter pontos de duas formas:

- a) Quando terminar o simulador, determinar com exatidão quantos marsamis estão em cada sala;
- b) Conseguir, no decorrer da simulação, em tempo real, identificar que, numa determinada sala num determinado instante, o número de marsamis “odd” e “even” são iguais.



Cada jogador é identificado pelo ID da equipe (inteiro)

Especificação

Descrição Geral

Está instalado um labirinto composto por salas unidas por corredores. No labirinto circulam marsamis. Existem sensores que detetam quando um marsami entra numa sala (vindo de um corredor) e sai de uma sala (indo para um corredor). Também existe um sensor de ruído que transmite o nível de ruído no labirinto no momento, e outro que transmite a temperatura. Para além de sensores que enviam informação, existem atuadores (atuam em função da informação que recebem) que permitem fechar e abrir os corredores das salas (impedindo os marsamis de circular) e ligar e desligar o ar condicionado.

Periodicamente são efetuadas “largadas” de marsamis. Cada marsami é colocado numa sala e, estes, por instinto natural, deslocam-se sempre entre salas até que o cansaço os faça parar (sobre o cansaço ver informação mais adiante) O objetivo é, no final de uma largada (quando eles estão todos quietos) cada jogador saber quantos ficaram em cada sala. Mas existem dois tipos de marsamis, “odd” e “even”. No final é necessário saber quantos “odd” e quantos “even” ficarem em cada sala.

Cada jogador, ao detetar que num determinado momento, numa sala, o número de “odds” é igual ao número de “evens”, pode acionar um “gatilho”. Se este “gatilho” for acionado antes que o equilíbrio entre “odds” e “evens” de desfaça, ganha 1 ponto, caso contrário perde meio ponto. Inicialmente todos os jogadores têm 0 pontos. Cada gatilho só pode ser accionado até 3 vezes por sala.

Cada jogador pode recorrer aos atuadores para fechar (ou abrir) um corredor ou, com apenas uma só acção fechar (ou abrir) todos os corredores em simultâneo. Note-se que se os corredores estiverem todos fechados os marsamis não se movem. Inicialmente os corredores estão todos abertos. Cada jogador pode ligar (diminui incrementalmente a temperatura da sala) ou desligar o ar condicionado do labirinto.

Se o nível de ruído e/ou temperatura exceder um certo limite, de forma a reduzir o ruído e/ou temperatura todas os corredores são automaticamente fechados e acaba a simulação. O jogador não pode voltar a abrir os corredores. O ruído aumenta proporcionalmente ao número de movimentos de marsamis. Se, por exemplo, alguns corredores forem fechados (fazendo diminuir o número de movimentos de ruído), o nível de ruído reduz-se.

Cada corredor une duas salas apenas. Vários corredores podem ir dar a uma mesma sala. O mecanismo que liga a sala ao corredor apenas permite que os marsamis circulem num sentido. Sempre que um marsami “entra” ou “sai” de uma sala é activado um sensor de passagem e não é possível dois marsamis saírem ou entrarem em conjunto (garante-se que uma ativação do sensor é para apenas um marsami).

Especificação (cont)

No início de cada largada os marsamis são distribuídos aleatoriamente por todas as salas. Posteriormente eles deslocam-se aleatoriamente pelos corredores que lhes estão acessíveis. Cada jogador tem o seu próprio labirinto, e no início a planta do labirinto é igual para todos os jogadores. Mas como os jogadores podem abrir e fechar corredores, as plantas vão divergindo de jogador para jogador. Sempre que um marsami atravessa um corredor a informação enviada pelo sensor está exemplificada nas seguintes duas mensagens :

{Player:10, Marsami:40, RoomOrigin:3, RoomDestiny: 5, Status:1}

“o marsami 40 ‘passou através de um corredor que o fez sair da sala 3 e entrar na sala 5”. O Player 10 colocado na mensagem serve para identificar o Jogador/Labirinto. O Status 1 não é gerado pelo sensor , é uma informação adicional colocada na mensagem para indicar que foi uma passagem “sem incidentes”.

{Player:5, Marsami:40, RoomOrigin:0, RoomDestiny: 3, Status:1}

Quando a sala de origem é “0” significa que estamos perante a largada inicial do marsami, ou seja, ele foi colocado na sala 3 no início da largada. Se quer a sala de Origem e a de Destino têm o valor 0, isso significa que o marsami está “preso” numa sala (última sala onde entrou), não pode sair (ou porque não tem nenhum corredor por onde sair – Status = 0 – ou por cansaço – Status = 2).

Utilizadores

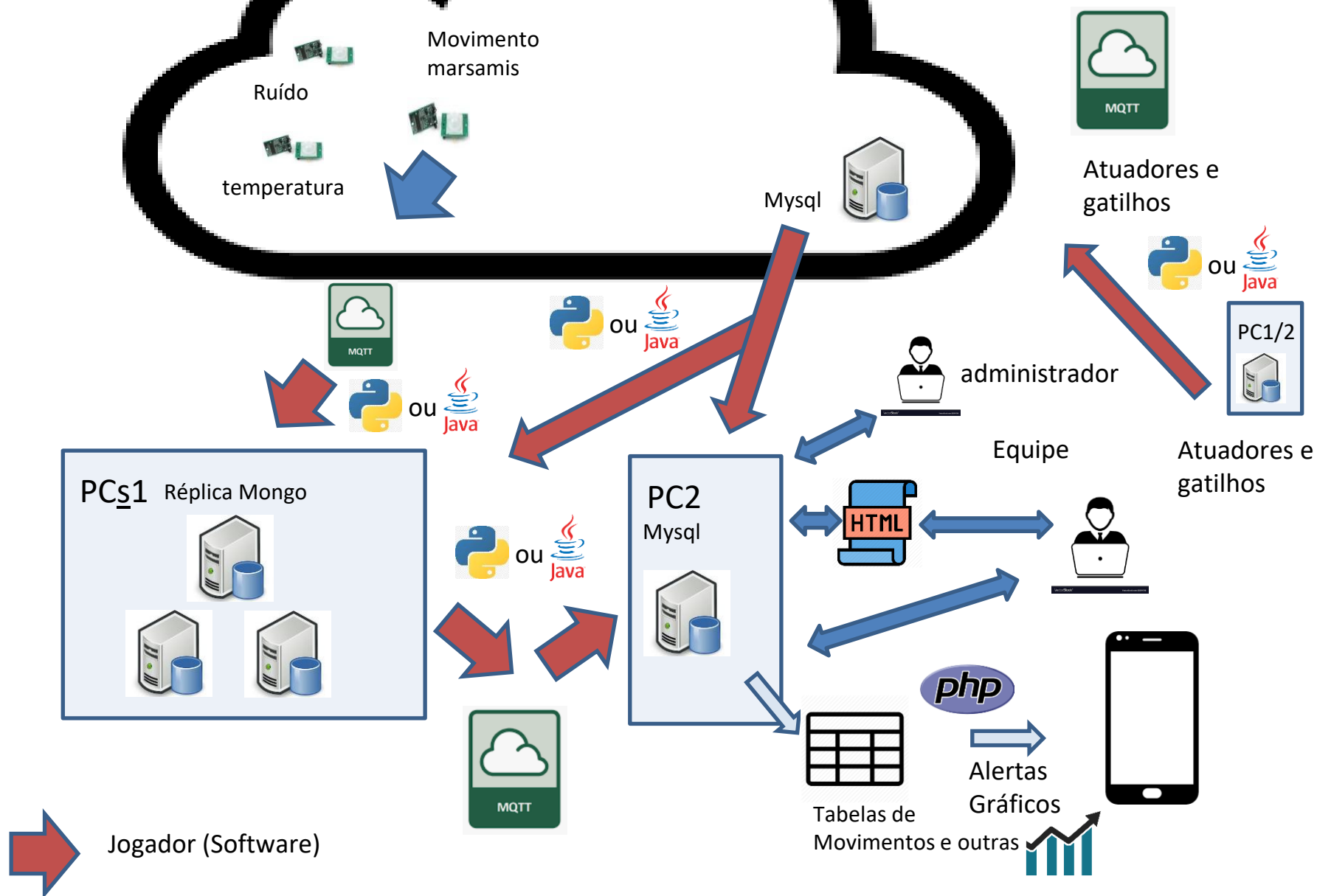
Existe um administrador da aplicação (não confundir com o administrador da base de dados) que cria e apaga utilizadores /equipes. Ele terá acesso a formulários próprios HTML/php que executarão procedimentos SQL. Estes formulários não são para implementar. As equipes (conjunto de utilizadores da mesma equipe) podem fazer a manutenção das simulações também através de formulários HTML/php. A manutenção consiste em criar e iniciar simulações e, para cada simulação, poder editar os parâmetros que achem relevantes criar na tabela simulação (ou em outras tabelas que decidam criar). Qualquer utilizador de uma equipe pode criar e iniciar uma simulação, mas apenas quem criou a simulação pode alterar. Apenas um utilizador pode alterar os seus dados pessoais.

As migrações das leituras dos sensores do MongoDB para o Mysql são feitas automaticamente e em tempo real mas, em caso de falha, o administrador da aplicação terá de ter uma forma de reinicializar o processo de migração (por exemplo, chamar um procedimento SQL ou um programa Java/Python/php).

Telemóvel Android

Cada equipe deverá ter a possibilidade de, através de um telemóvel, monitorizar através de um gráfico a evolução do número de marsamis por sala, receber eventuais alertas de perigo de excesso de ruído e/ou temperatura., e visualizar graficamente a evolução da temperatura e ruído.

Arquitetura



Labirinto Notas

Formulários HTML

Os formulários para o administrador não são para serem implementados.

Para os utilizadores da equipe apenas é necessário:

- Fazer login
- Criar (ou seleccionar um de uma lista de simulações seus para alterar) uma simulação (ou configuração de simulação) (tabela simulação e outras criadas pelo grupo) e, para essa simulação , editar os valores associados. Quando está a alterar não pode alterar valores de chaves estrangeiras e primárias. Não é suposto editar uma simulação quando ele está a decorrer. Todo os outros acessos são feitos diretamente na base de dados (ou diretamente ou SPs, decisão do grupo excepto se SPs obrigatórios).
- Botão “Iniciar” para iniciar uma simulação (por exemplo, php que chama um executável python ou batch file no computador local.

Diversos

- Apenas os utilizadores da equipe podem editar na base de dados a informação (campos) das simulações que criaram, quer diretamente na BD quer através de SPs (chamados pelo php ou chamados diretamente na bd).

Início e Fim de Simulação

A simulação começa quando o grupo o iniciar (por um ou vários programas python ou html). Termina automaticamente quando todos os corredores têm o status “2” / cansaço (note-se que quando os corredores se fecham porque foi atingido o limite de temperatura ou ruído, os marsamis ficam imóveis até se “cansarem”).

Os dados dos sensores de ruído e temperatura apenas necessitam ser armazenados enquanto decorre a experiência.

No telemóvel aparece um gráfico que mostra para cada sala quantos marsamis lá estão (odd e even), gráfico do ruído, gráfico da temperatura.

Bases de Dados Local (PC2)

Simulacao	MediçõesPassagens
<u>IDSimulacao (autoincrement)</u>	<u>IDMedição (autoincrement)</u>
Descrição (text)	Hora (timestamp)
Equipa (int)	SalaOrigem (int)
DataHorainicio (timestamp)	SalaDestino (int)
	Marsami (int)
	Status (int)
Utilizador	Mensagens
Nome (varchar 100)	<u>ID (autoincrement)</u>
Telemovel (varchar 12)	Hora (timestamp)
Tipo (Varchar 3)	Sala (int)
<u>Email (varchar 50)</u>	Sensor (varchar 10)
DataNascimento (date)	Leitura (decimal 6,2)
Equipa (int)	TipoAlerta (varchar 50)
Temperatura	Msg (varchar 100)
<u>IDTemperatura (autoincrement)</u>	HoraEscrita (timestamp)
Hora (timestamp)	
Temperatura varchar(12)	OcupacaoLabirinto
	<u>IDJogo (int)</u>
Som	NumeroMarsamisOdd (int)
<u>IDSom (autoincrement)</u>	NumeroMarsamisEven(int)
Hora (timestamp)	<u>Sala (int)</u>
Som varchar(12)	
TabelasAdicionais....	

Valores válidos 1 ()temperatura) e 2 (Ruído)

Valor de temperatura ou som enviado pelo sensor

Hora em que foi feito o insert, não necessariamente igual à hora em que a medição que gerou a mensagem foi registada.

Pode ser alterada pelo grupo, com mais tabelas, novos campos nas tabelas existentes, nomeadamente para guardar mais informação.

As tabelas a amarelo são lidas pelo android, o que for alterado tem de ser alterado nos scripts/android. A tabela Ocupações Labirinto indica no momento quantos marsamis estão em cada sala. Na tabela mensagens todos os campos são opcionais.

Bases de Dados Mysql Nuvem

Base de Dados sempre online, pronta a ser consultada (apenas read only)

Corridor	
<u>RoomA (int)</u>	
<u>RoomB (int)</u>	
SetupMaze	
numberrooms	número de salas
numbermarsamis	número de marsamis
numberplayers	<i>número de jogadores (irrelevante para as equipas)</i>
normaltemperature	temperatura do labirinto vazio
temperaturevarhightoleration	variação positiva face ao normaltemperatura a partir do qual o labirinto fecha
temperaturevarlowtoleration	variação negativa face ao normaltemperatura a partir do qual o labirinto fecha
normalnoise	som do labirinto vazio
noisevartoleration	variação face ao normalsound a partir do qual o labirinto fecha

Os valores não são alterados a meio de uma simulação.

Database maze

Host: 194.210.86.10 Username/Senha: aluno/aluno

mysql --user=aluno --password=aluno --host 194.210.86.10 maze

Interagir Labirinto (Servidor)

Sensores (receber mensagens)

1) Sensores de ruído

mqtt_topic=pid_mazesound_*n* (em que *n* é o numero do jogador)

{Player:11, Hour: "2024-07-04 16:29:21.281898", Sound: 19.0}

2) Sensores de temperatura

mqtt_topic=pid_mazetemp_*n* (em que *n* é o numero do jogador)

{Player:11, Hour: "2024-07-04 16:29:21.281898", Temperature: 19.0}

3) Sensores de movimento

mqtt_topic=pid_mazemov_*n* (em que *n* é o numero do jogador)

{Player:1, Marsami:47, RoomOrigin:4, RoomDestiny: 5, Status:1}

Atuadores

Mqtt_topic="pid_mazeact"

{Type: Score, Player:1, Room: 2}

{Type: OpenDoor, Player:1, RoomOrigin: 3, RoomDestiny: 2}

{Type: CloseDoor, Player:1, RoomOrigin: 3, RoomDestiny: 2}

{Type: CloseAllDoor, Player:1}

{Type: OpenAllDoor, Player:1}

{Type: AcOn, Player:1}

{Type: AcOff, Player:1}

Iniciar Simulação/Labirinto

C:>mazerun.exe *numero_grupo*

Existem parâmetros opcionais: '—flagMessage —delay —broker —portbroker

delay é im inteiro e repreenta o número de segundos entre cada movimento dos marsamis (número elevado significa que os marsamis movem-se lentamente)

flagMessage – permite seleccionar as mensagsns que são enviadas para a consola: 1 mostra tudo; 2 apenas movimentos; 3 apenas Som; 4 apenas atuadores; 5 apenas temperatura

Se o servidor mqtt estiver lento podem usar outro, passando-o como argumento junto com o porto. Por exemplo:

C:> mazerun.exe 30 —broker *broker.hivemq.com* —portbroker 1883

MQTT Brokers

Escolher um que esteja onine e rápido, exemplos:

cloud_server=tcp://broker.mqtt-dashboard.com:1883

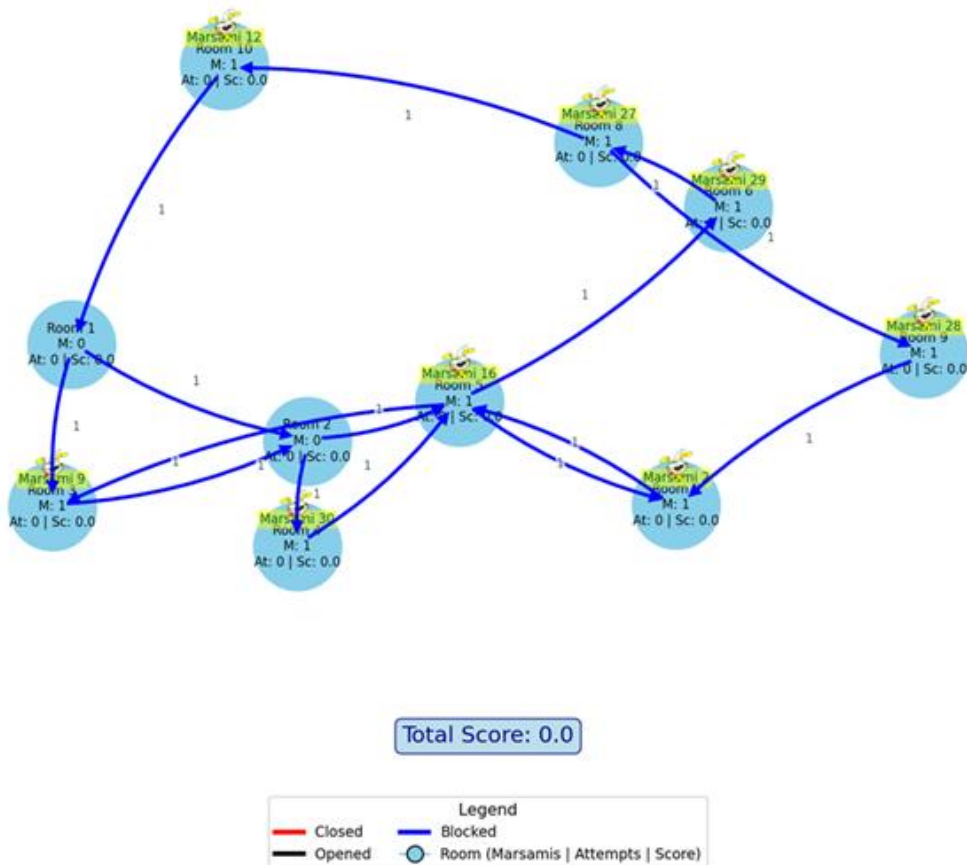
cloud_server=tcp:// *broker.hivemq.com* 1883

Monitorizar Labirinto

Movimentos Marsamis

C:> monitmaze.exe n

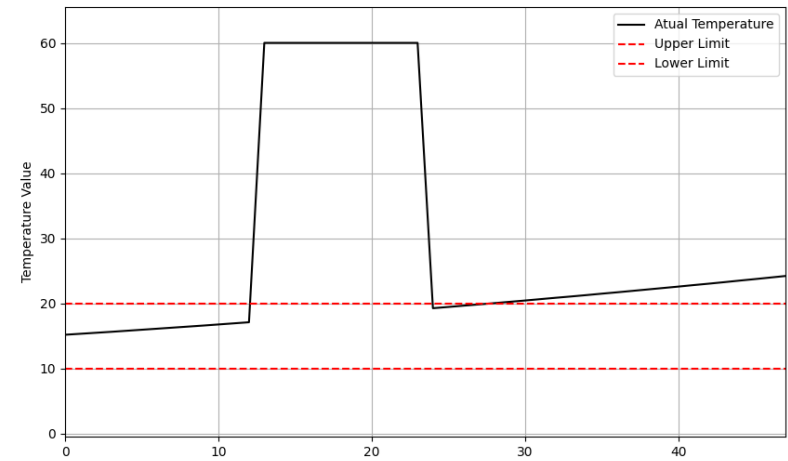
Maze for Team 50



Temperatura

C:> monittemp.exe n

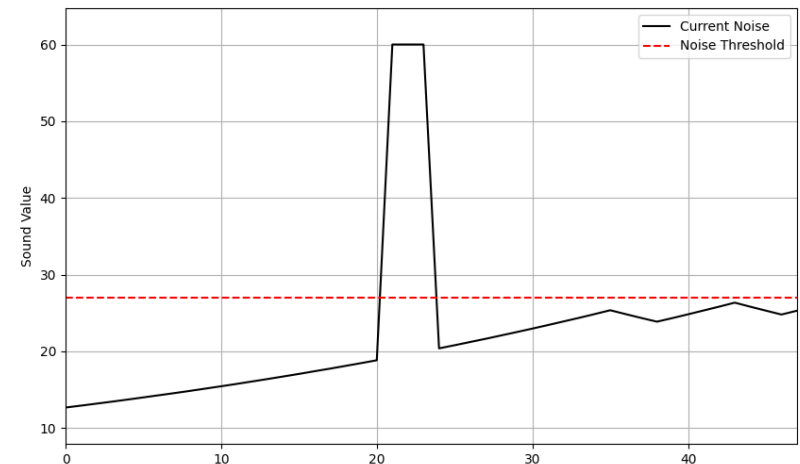
Temperature Evolution for team 50



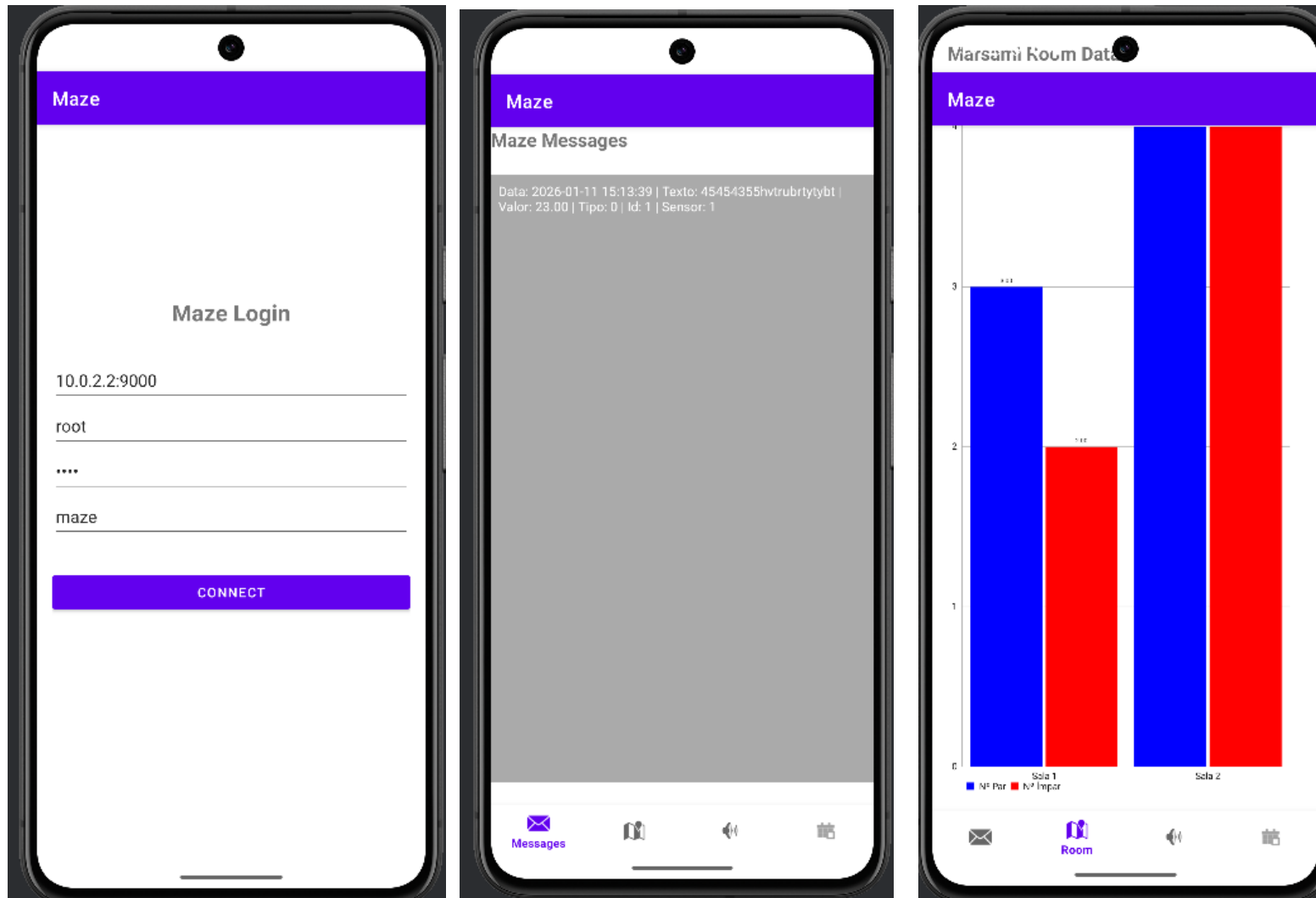
Ruído

C:> monitsound.exe n

Noise Evolution for team 50

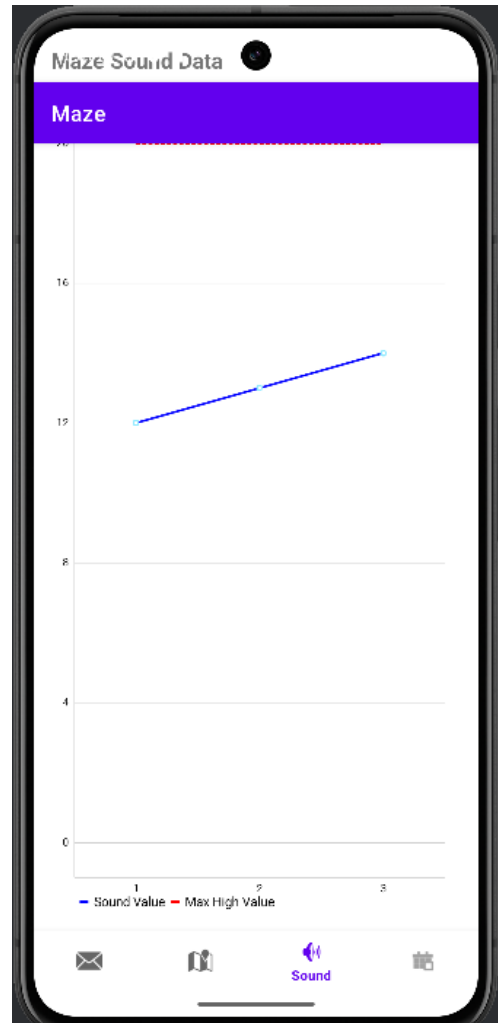
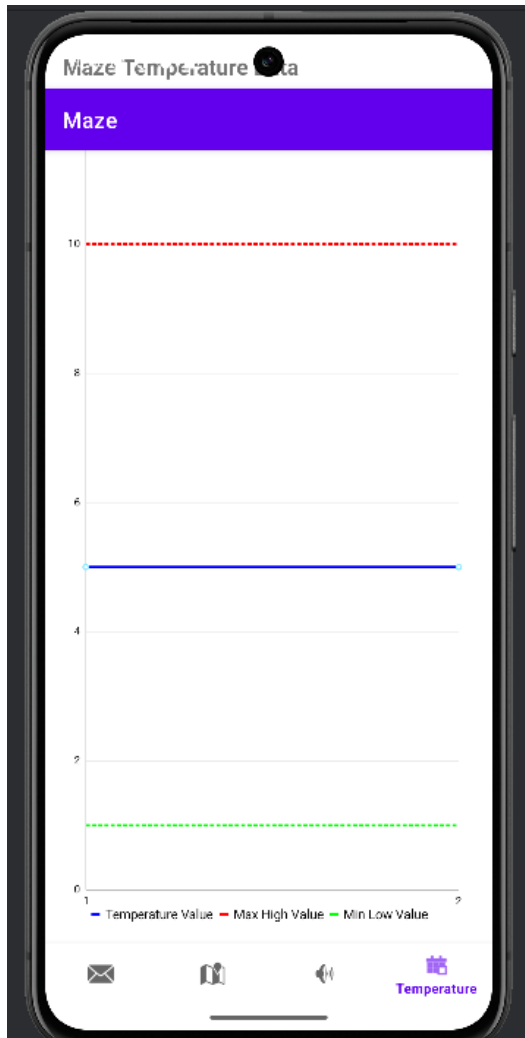


Interação Android



Valores não reais, apenas para
mostrar layout de gráfico

Interação Android



Valores não reais, apenas para mostrar layout de gráfico